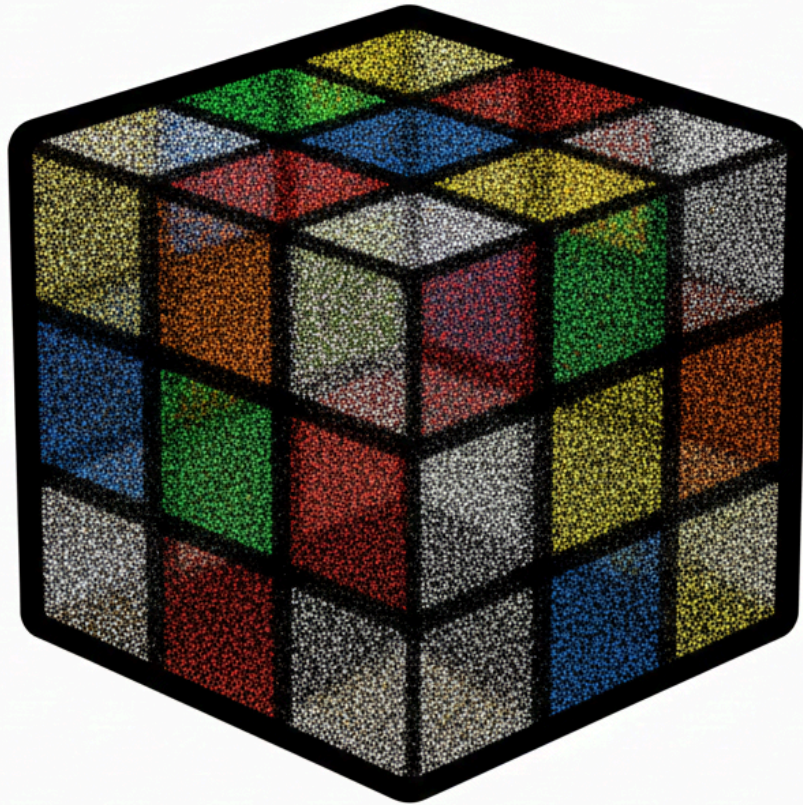


Assignment 3 – Rubik's Cube + PointNet



Task 1 - Rubik's Cube

Overview:

In this task you will render a dynamic Rubik's cube using **OpenGL** in C++.

Part 1 – building the Rubik's cube scene:

1. Create a **Perspective View Camera** with a **FOV Angle** of 45.
2. Build a **Cube** mesh with the **plane.png Texture** to render a single cube.
See what happens when you translate and rotate the cube.
3. For building the Rubik's cube you will need to duplicate the cube 26 or 27 times and translate each cube to its initial position.

Part 2 – data structures:

1. Build data structures that will hold information about the current position of each cube on the full cube based on their shape index.
You will need to know which of the cube you need to rotate each time you rotate a wall of the Rubik's cube.
2. You may build other data structures to hold for each cube its current rotations and translation.
This data structure will save the transformations you did for each cube.
(But it is possible to solve this task without them).
3. Build data structure (class) to hold all the Rubik's cube data.
(It is also possible to store all the data on the available classes in the engine if you find it easier).

Part 3 – rotations in a row (the difficult part):

1. Keyboard keys callback function:

- a. "R" – press state for **right** wall rotation (90 degrees clockwise).
- b. "L" – press state for **left** wall rotation (90 degrees clockwise).
- c. "U" – press state for **up** wall rotation (90 degrees clockwise).
- d. "D" – press state for **down** wall rotation (90 degrees clockwise).
- e. "B" – press state for **back** wall rotation (90 degrees clockwise).
- f. "F" – press state for **front** wall rotation (90 degrees clockwise).
- g. " " (Space) – press state for **flipping** rotation direction.
(from clockwise to counterclockwise or vice versa).
- h. "Z" – press state: dividing rotation angle by 2 (until minimum of 90).
- i. "A" – press state: multiply rotation angle by 2 (until maximum of 180).

j. **Notice:** The rotation directions are only relative to the cube **initial** camera view.

2. Mouse keys callback function:

- a. Moving while holding **left button** will **show** rotation of the whole **Rubik's cube**:
Left to right movement will rotate the cube around the **global Y** axis and up to down movement will rotate it around **global X** axis.
- b. Moving while holding the **right button** will move the **camera** up, down, left or right.
- c. **Scrolling** up and down will move the camera along the **Z axis** backward and forward
(Make sure that the cube is moving backward and forward from the current view).

d. **Notice:** Implementing all the transformations above can be done by updating the **View Matrix** only.

3. Color Picking:

- a. The Picking Mode should get enabled and disabled by pressing the "P" key.
- b. Picking a **specific** cube with the mouse, to **translate** and **rotate** it, should be possible.
- c. When translating the picked cube with the mouse **right** click, the cube should stay under the mouse.
(Implement the **Z-Buffer** formula to make it work).
- d. When rotating the picked cube with the mouse left click, the cube should be rotated according to the current camera view.

e. **Notice:** Implementing all of the transformations above should be done on a cube's **Model Matrix**.

Part 4 – cube functionality:

- The cube structure should never "**break**" after multiple different wall rotations.
- In the "Color Picking" mode, breaking the cube is **required**, and when exiting this mode, you are allowed to reload your whole program.

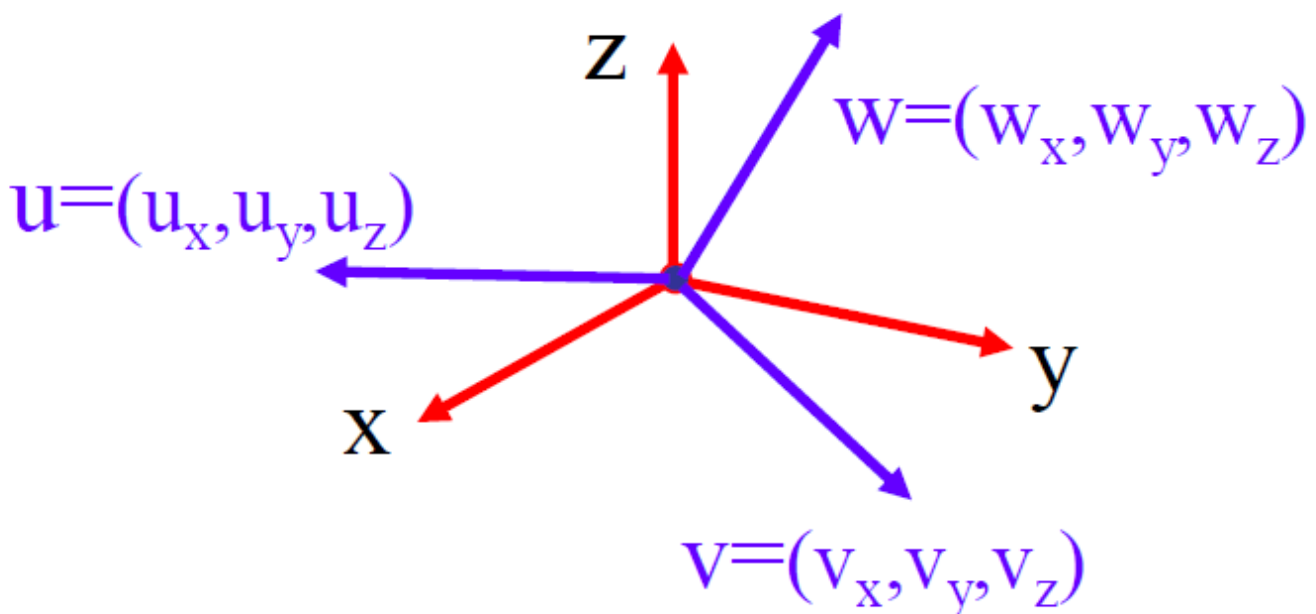
Color Picking Guidance:

You may choose in which way you want to implement the Color Picking method, but here are some suggested steps to do it: by modifying the MouseButtonCallback:

1. Modify the **Shaders** to support Picking Mode, so that in this mode the shape will get their colors only from the uniform **u_Color** parameter.
2. Modify the **MouseButtonCallback** to do as follows:
 - a. Clear the window.
 - b. Get the: VBO, VAO, Shader and the MVP matrices of all the cubes you created.
 - c. Bind the: VBO, VAO and the Shader.
 - d. Draw each one of the cubes with a unique color in **u_Color** that will be passed to the Shader.
 - e. Use the **glReadPixels** function to read the **color** in the selected pixel and convert the color to the relevant cube index and keep it.
 - f. Use the **glReadPixels** function to read the depth in the selected pixel (to use for the Z-Buffer) and keep it.
3. Modify the **CursorPosCallback** to check if a cube was selected, and supported the Translate and Rotate operation based on the selected Mouse Button.

General tips:

1. When calculating the Model Matrix per cube, set the Translate, Scale and Rotate Matrices as you see fit:
 - a. $rot \times trans \times scl$ – For rotations around the global center. (Better for this task)
 - b. $trans \times rot \times scl$ – For rotations around the object center.
2. First, try to implement one wall rotation in the Rubik's cube.
3. Plan where each cube supposes to be after every rotation and how you can follow each cube (hint: use index array).
4. Then try to implement a support for multiple different rotations in a row.
Remember that after each wall rotation the rotated cubes' coordinate system is getting changed.
Use the equations for **Changing Coordinate Systems** to handle that:



Task 2 - PointNet + ShapeNet

Overview:

In this task you will analyze the AI Model **PointNet**, set up the 3D Dataset **ShapeNet** (of shapes in point cloud), and run a **Jupyter Notebook** in Python.

Part 1 – Project setup:

1. Create a new folder in the project (for example: “PointNet”) and download the following files from Kaggle:
 - a. <https://www.kaggle.com/code/jeremy26/pointnet-shapenet-dataset> - PointNet AI Model
(Put in path: “/PointNet/pointnet-shapenet-dataset.ipynb”)
 - b. <https://www.kaggle.com/datasets/jeremy26/shapenet-core-seg> - ShapeNet Dataset
(Put in path: “/PointNet/shapenet-core-seg/<unzipped-content>”)
 - c. <https://www.kaggle.com/datasets/jeremy26/lidar-od-scripts> - Utility Scripts
(Put in path: “/PointNet/lidar-od-scripts/<unzipped-content>”)
2. Install **VS Code** extensions for: **Python** and **Jupyter**.
3. Create a Python environment (Suggest version: **Python 3.10.x**)
(See the following guide to setup a python environment: <https://www.youtube.com/watch?v=D2cwvpJSBX4>)
4. Download the following requirements libraries to your Python environment:

```
Python
numpy>=1.21.0
torch>=1.9.0
matplotlib>=3.4.0
plotly>=5.0.0
tqdm>=4.62.0
```

5. Update the paths on the notebook to the paths on your computer.

Part 2 – Dataset preparation:

Training on the full ShapeNet dataset requires a strong GPU, and on CPU it might take forever.

Therefore, you are required to create a smaller **subset** of the dataset. To do that, follow these steps:

1. Create a subset folder structure:
Create a copy of the original ShapeNet dataset folder structure, but include **only 5 to 6 selected categories** (that has at least 100 samples, like Airplane with 1958 samples).
(The file `synsetoffset2category.txt` contains the mapping between folder names and shape categories)
2. Select samples per category:
For each selected category, randomly select **100 to 150 samples**, where each sample consists of a matching pair of: a point cloud file (**.npy**) and its corresponding label file (**.seg**).
Copy only the selected pairs into your subset folders.
3. Update train / validation / test splits:
Modify the dataset split JSON files (`train`, `val`, `test`) so that:
 - a. The split ratio is 70% (train), 10% (val), 20% (test).
 - b. The split is maintained **per category** and **for the entire subset dataset**.
For example: 70 train, 10 val, 20 test shapes (out of 100) for the **Airplane** category.
And: 350 train, 50 val, 100 test shapes (out of 500) for the **entire subset**.
4. Training guidance:
After creating the subset, train the PointNet model using this dataset and analyze its results.
You may **increase the number of training epochs** to improve model accuracy, keeping in mind that higher epochs increase training time.

Part 3 – Analysis tasks:

For each selected task, you may either create a new notebook or continue working after the end of the main notebook, and test both the **Classification** and **Segmentation** models when applicable (unless specified otherwise). When “**analyze the results**” is mentioned, you should show: **a single sample visual result**, **the NLL loss** (like in the original notebook) and (new) **the accuracy** you get per model (Correct results divided by the Total). Then, **write a short summary of your findings** on this task.

Select 2 out of the 3 following task:

- **3.1. Task:** Understand **ShapeNet features importance for PointNet**
 1. Test the effectiveness of modifying the data dimensions and features:
 - Clone 2 times the ShapeNet subset you have created.
 - In Subset 1: For each sample randomly drop a single dimension (e.g., **Orthographic Projection** on (x, y) or (y, z)).
 - In Subset 2: For each sample, apply a random rotation and then apply **Perspective Projection** on a random dimension ([See video](#) for example).
 - **Train + Test the PointNet on each subset separately, and analyze the results.**
- **3.2. Task:** Understand **PointNet architecture modifications**
 1. Analyze the **impact of architecture on feature learning** by implementing the following changes:
 - Observe **Global pooling (max) vs Local aggregation (avg)**
 - Original PointNet uses **MLPs (64, 128, 1024)** → max pooling → fully connected classifier.
 - Change **number of neurons** (e.g., 32, 64, 512)
 - Add **extra hidden layer**
 - Replace **ReLU with LeakyReLU**
 - Change **pooling** from max to average
 - **Train + Test the Modified PointNet on Part 2 subset, and analyze the results.**
 2. Analyze the T-Net learned transformations:
 - Choose a class and 3 (or more) different samples from it.
 - Display for each sample its original transformation vs the 2 PointNets' T-Net transformations (**Original vs Original PointNet vs Modified PointNet**).
 - **Notices:** You need to apply the **T-Net** transformations on the shapes and then display them.
- **3.3. Task:** Understand **PointNet's analysis w.r.t. sampling**
 1. **Notice:** Since 5-6 shapes are distributed equally, you don't have to select specific categories here.
 2. Observe limits of robustness for **sparse/thin shapes**
 - Clone **6 times** the ShapeNet subset you have created (See **Downsampling** on the next page).
 - In Subsets 1, 2, 3: **Drop 10%, 30%, 50% of points** using **Voxel Downsampling (*)**.
 - In Subsets 4, 5, 6: **Drop 10%, 30%, 50% of points** by **Uniform Downsampling (**)**.
 - **Notice:** The original point files contain approximately **2500 points**.
After downsampling, adjust the number of points sampled per shape accordingly (For example, set **2250** points to sample, for a **10%** drop of points).
 - **Train + Test the PointNet on each subset separately, and analyze the results**
(Here you can skip **downsampling** the `.seg` files, and testing the **Segmentation Model**).
 3. **Pooling analysis**
 - Compare **max pooling** vs **average pooling** for handling missing points.
 - PointNet uses **global max-pooling** over points.
 - You can:
 - Replace with **average pooling**
 - Replace with **top-k max pooling** (e.g., top 5 points)
 - **Train + Test the PointNet on each pool and subset combination, and analyze the results.**

(*) Voxel Downsampling: A technique to reduce the number of points on a selected shape in point cloud format with minimal structural loss.

The key idea is to:

1. Setup a 3D grid on the points world and mark each occupied cell (voxel) by a point with a bit of 1.
2. Convert the grid back to points by converting each occupied cell (i,j,k) indices to (x,y,z) point coordinates.
(Notice: This idea resembles the “Minification” algorithm as you will learn in the “Textures” lecture).

To implement this correctly you need to perform the following steps:

1. Translate all the points so that their coordinates will be positive.
2. Apply the Grid on the points and mark occupied cells (voxels).
3. Convert back the occupied cells to 3D point coordinates.
4. Translate back the points to their original center (by applying the opposite Translation from step 1).

() Uniform Downsampling:** A technique to reduce the number of points on a selected shape in point cloud format by randomly selecting which points to keep (structural loss can be high).

Common approaches are:

1. Selecting every K point (to drop or to keep) - For example K=2 will drop 50% of the points.
2. Random selection - Count the number of points and randomly select how many points to drop.

You may choose either way you like.

Assignment Score:

Rubik’s Cube (50 points):

- **Sanity** - Rendering the Rubik’s cube with a Perspective Camera: **10 points**
- Full support for multiple wall rotations (Clockwise/Counterclockwise) of the Rubik’s cube: **10 points**
- Option to 180 degrees rotations: **5 points**
- **(Full Rubik's Cube)** Rotation according to camera view: **5 points**
- **(Full Rubik's Cube)** Translation up/down/left/right according to camera view: **5 points**
- **(Full Rubik's Cube)** Scroll in and out according to camera view: **5 point**
- **(Color Picking)** Z-Buffer translations for a selected cube: **5 points**
- **(Color Picking)** Rotations for a selected cube: **5 points**

PointNet (50 points):

- **Sanity** - Project setup and notebook run: **10 points**
- Dataset preparation: **10 points**
- Analyze tasks: **30 points**

Submission:

Submit zip file with the following files:

1. Link to your [GitHub](#) repository.
2. **(Optional)** Text, Doc or PDF file with short explanation about the changes you did in the engine (files you change and functions you modify).

This file will help you and us to understand what changes you have made in the engine to complete the assignment 😊

The zip file name must include your ID numbers as follows: **ID1_ID2.zip**.